

CUADERNOS DE COMPILADORES PARA APRENDIZAJE AUTOMÁTICO

Tutorial de tinygrad

Del tensor al programa ejecutado

Material para quien ha completado *Compiladores de ML desde cero*. Explicaciones paso a paso, programas completos, ejercicios con solución y validación local en CPU o GPU según la práctica.

Kripta Studios

Edición del 1 de octubre de 2026

Material didáctico independiente. Los programas originales y las evidencias acompañan al PDF en el repositorio. Las APIs externas se fijan a las versiones indicadas.

CAPÍTULO 1

De Lumbre a tinygrad

En el libro principal construiste un compilador que convierte operaciones tensoriales en programas nativos. `tinygrad` permite estudiar esas mismas responsabilidades en un proyecto externo: construir un grafo, diferenciarlo, planificar trabajo, generar código y ejecutarlo. En este tutorial comenzarás por resultados pequeños que puedes calcular a mano.

No necesitas conocer PyTorch. Usaremos Python, NumPy como referencia y las ideas de tensores, gradientes y memoria que ya aprendiste. Cada práctica tiene un contrato, un programa ejecutable y una comprobación. Cuando aparezca una API propia de `tinygrad`, explicaremos qué papel cumple en el experimento.

1.1 Una revisión concreta

Esta edición utiliza el commit `c3aec477b99d9bb87c54d91897cf60acd3f17441`, del 30 de septiembre de 2026. Su paquete declara la versión 0.14.0. El hash identifica el código exacto; el número de versión por sí solo no distingue todos los estados de desarrollo.

La interfaz de esta revisión difiere de ejemplos antiguos: el constructor de `Tensor` no recibe `requires_grad`, el entrenamiento del optimizador se activa con `Context(TRAINING=1)` y la selección de renderer utiliza `DEV`. Seguiremos las firmas del código fijado, no una mezcla de tutoriales de distintas fechas. [2, 3, 5]

1.2 La correspondencia que vamos a estudiar

Lo que ya conoces	Dónde lo buscarás en <code>tinygrad</code>
Tensor y grafo de operaciones	<code>Tensor</code> y su atributo <code>uop</code> .
Cálculo del resultado	<code>realize</code> , <code>numpy</code> e <code>item</code> .
Gradientes	<code>backward</code> , <code>gradient</code> y el atributo <code>grad</code> .
Actualización de pesos	Optimizadores de <code>tinygrad.nn.optim</code> .
Plan y ejecución	<code>schedule</code> , <code>codegen</code> , <code>renderer</code> y <code>runtime</code> .
Repetición de un programa	<code>TinyJit</code> .

Las correspondencias ayudan a orientarte, pero no afirman que las representaciones internas de Lumbre y `tinygrad` sean intercambiables. Un cambio en el grafo puede llevar a planes distintos aunque la fórmula de partida sea la misma.

Punto de entrada

Calcula la forma de sumar $[2,3]$ con $[3]$ y reducir el eje 1. La suma tiene forma $[2,3]$ y el resultado de la reducción tiene forma $[2]$. Esa será nuestra primera comprobación.

CAPÍTULO 2

Instalar sin perder la versión

2.1 Crear un entorno del tutorial

Trabajaremos dentro de Linux o WSL. Necesitas Python 3.11 o posterior, un entorno virtual funcional y un compilador del backend elegido. Para reproducir esta edición, clona el repositorio de tinygrad fuera del código de Lumbre y selecciona el hash:

```
python3 -m venv .venv-tinygrad
source .venv-tinygrad/bin/activate
git clone https://github.com/tinygrad/tinygrad.git /tmp/tinygrad-course
git -C /tmp/tinygrad-course checkout \
    c3aec477b99d9bb87c54d91897cf60acd3f17441
python -m pip install /tmp/tinygrad-course numpy==2.3.5
```

El directorio de clonado debe estar libre. Si `venv` informa de que falta `ensurepip`, instala el paquete de entornos virtuales de tu distribución o utiliza `uv venv`. No continúes hasta tener un intérprete que pueda importar tinygrad y NumPy.

2.2 CPU y CUDA son decisiones explícitas

Los programas reciben `-device CPU` o `-device CUDA`. La entrada se crea en ese dispositivo y la salida del informe registra lo solicitado. Evitamos dejar esta elección a la detección automática porque queremos comparar ejecuciones identificables.

En la revisión fijada, `DEV='CPU:CLANG;CUDA:NVCC'` selecciona Clang para el backend CPU y nvcc para CUDA. Las dos rutas necesitan sus herramientas en `PATH`. En CUDA también debe ser visible la biblioteca del controlador; WSL suele ofrecerla en `/usr/lib/wsl/lib`. Estas variables se establecen antes de iniciar Python.

```
export DEV='CPU:CLANG;CUDA:NVCC'
export CC=clang-18
python tutorials/tinygrad/labs.py --device CPU \
    --output tutorials/reports/mi_tiny_cpu
python tutorials/tinygrad/labs.py --device CUDA \
    --output tutorials/reports/mi_tiny_cuda
```

El backend CUDA utiliza la interfaz del controlador NVIDIA. tinygrad también tiene otras rutas de dispositivo; este tutorial valida solo las dos nombradas. No interpretes la presencia de un backend en el repositorio como una prueba de funcionamiento en tu equipo.

2.3 Leer la instalación antes de ejecutar

En una terminal Python puedes consultar `tinygrad.__file__` para localizar el paquete importado. En el clonado, `git rev-parse HEAD` muestra la revisión del fuente. Conserva ambos datos: un clonado correcto no garantiza que otro entorno Python importe esa instalación.

El tutorial no instala `tinygrad` dentro del entorno de Lumbre. Mantener dos entornos permite reproducir cada experimento y cambiar una dependencia sin alterar la otra campaña.

CAPÍTULO 3

Tensores: datos, forma y realización

3.1 La primera fórmula

Creamos un array con dos filas: $[0,1,2]$ y $[3,4,5]$. Sumamos el bias $[0.5,-1,2]$, aplicamos ReLU y sumamos cada fila. El resultado esperado es $[4.5,13.5]$. La primera fila después de ReLU es $[0.5,0,4]$; la segunda es $[3.5,3,7]$.

```
a = np.arange(6, dtype=np.float32).reshape(2, 3)
x = Tensor(a, device=device)
bias = Tensor([0.5, -1.0, 2.0], device=device)
y = (x + bias).relu().sum(axis=1)
print(y.numpy())
```

El broadcasting reutiliza el bias para ambas filas. No lo confundas con un parámetro que cambie por fila. Al especificar FP32 en los datos de NumPy, evitamos que una referencia use por accidente FP64 mientras el programa utiliza otra precisión.

3.2 Describir y ejecutar

Una expresión tensorial puede construir trabajo pendiente. `realize()` solicita su ejecución y devuelve el tensor. `numpy()` entrega un array en el host y requiere disponer del resultado. `item()` extrae un escalar; si lo invocas dentro de cada iteración GPU, introduces una observación del host que puede afectar al tiempo.



Realizar el mismo tensor ya calculado no representa una nueva multiplicación. Para medir una operación repetida necesitas construir el trabajo adecuado o utilizar una función capturada que se vuelva a ejecutar. Esta distinción evita medir cien lecturas de un resultado como si fueran cien kernels.

3.3 Transponer sin perder el contrato

La práctica comprueba también `x.transpose()` contra `a.T`. La forma cambia de $[2,3]$ a $[3,2]$. Que una transposición pueda describirse mediante índices no garantiza que todas las operaciones posteriores eviten una copia; la planificación depende de los consumidores y del backend.

Antes de forzar una materialización contigua, pregunta qué operación la necesita y mide su coste. Un movimiento lógico y una transferencia entre dispositivos son decisiones diferentes.

Evitar una referencia circular

Escribe el resultado esperado de la primera fórmula como una lista literal y compáralo con la salida. Después cambia el bias. Si construyes la referencia con la misma expresión de tinygrad, podrías repetir el mismo fallo en los dos lados.

CAPÍTULO 4

Gradientes que puedes comprobar a mano

4.1 Una pérdida de cuatro variables

Sea $L(x) = \frac{1}{4} \sum_i x_i^2$ para $x = [-2, -0.5, 1, 3]$. La derivada es $x_i/2$, de modo que esperamos $[-1, -0.25, 0.5, 1.5]$. Esta cuenta no depende de cómo planifique los kernels el compilador.

```
x = Tensor(np.array([-2, -0.5, 1, 3], dtype=np.float32),
            device=device)
loss = (x*x).mean()
loss.backward()
print(x.grad.numpy())
```

En el commit fijado, `backward` rellena gradientes de tensores flotantes vivos que participan en el grafo. `is_param` sirve para la selección de parámetros del optimizador; no sustituye la explicación de cómo se construye el backward. Lee la implementación antes de adaptar un ejemplo escrito para una API anterior.

4.2 Dos comprobaciones independientes

La práctica compara con la fórmula analítica y con diferencias finitas centrales en NumPy FP64. Perturbamos una coordenada cada vez y calculamos:

$$g_i \approx \frac{L(x + \varepsilon e_i) - L(x - \varepsilon e_i)}{2\varepsilon}.$$

Usamos $\varepsilon = 10^{-3}$. Para esta función cuadrática la comparación es sencilla. En funciones con cancelación numérica o puntos no diferenciables necesitarías elegir entradas y tolerancias con más cuidado.

4.3 Acumulación y grafos retenidos

Los gradientes son tensores y también pueden representar cálculo pendiente. Un segundo `backward` puede acumular sobre `grad`. Para una nueva actualización, el optimizador pone los gradientes a `None` mediante `zero_grad`; después construimos una pérdida nueva.

No guardes todos los tensores de pérdida si solo quieres una historia de números. En la práctica de entrenamiento guardamos valores escalares. Así el registro no conserva por accidente una cadena de grafos que ya no necesitamos.

Resolver una modificación

Si sustituyes `mean()` por `sum()`, la derivada debe multiplicarse por cuatro. Predice `[-4,-1,2,6]` y modifica tanto la fórmula analítica como la referencia de diferencias finitas. Una prueba que no cambia su expectativa debe fallar.

CAPÍTULO 5

Entrenar una regresión y guardar sus pesos

5.1 Un problema con respuesta conocida

Generamos 33 valores de x entre -1 y 1 y definimos $y = 3x - 0.5$. El modelo es $\hat{y} = xw + b$, con w y b inicializados a cero. Usamos todos los ejemplos en cada paso y una pérdida cuadrática media. No hay descarga, muestreo aleatorio ni conjunto de evaluación oculto.

Esta práctica comprueba actualización y serialización. No evalúa generalización: la relación de salida la hemos fijado y conocemos la solución.

5.2 Seguir una actualización

El optimizador recibe solo $[w, b]$, no los datos. La tasa de aprendizaje vale 0.2. Abrimos `Context(TRAINING=1)`, borramos gradientes, calculamos la pérdida, propagamos y actualizamos. Repetimos cien veces.

```
optimizer = SGD([w, b], lr=0.2)
with Context(TRAINING=1):
    for _ in range(100):
        optimizer.zero_grad()
        loss = ((x @ w + b - y)**2).mean()
        history.append(float(loss.item()))
        loss.backward()
        optimizer.step()
```

Extraemos la pérdida antes de actualizar porque queremos registrar el valor correspondiente a esos pesos. El registro de un paso y la predicción final no son necesariamente el mismo instante del entrenamiento.

5.3 Derivar el primer paso

Con pesos cero, el error es $-3x + 0.5$. Como las entradas son simétricas, su media es cero. El gradiente de b es 1, por lo que el primer valor nuevo de b es -0.2. El gradiente de w es $-6 \text{mean}(x^2)$. Puedes calcular esa media a partir de los 33 puntos y comprobar el signo de la actualización.

Si b se mueve hacia valores positivos, revisa el signo de la pérdida y de la actualización. Si los pesos no cambian, comprueba que estén en la lista del optimizador y que `step()` se ejecute.

5.4 Guardar, cargar y saber qué falta

La práctica escribe `w` y `b` en `regression_weights.npz`, crea tensores nuevos con esos arrays y comprueba identidad de las predicciones recargadas. El archivo contiene pesos para inferencia. No afirma conservar el estado de un optimizador general, el RNG o una posición de lectura de datos.

Para reanudar Adam necesitarías también momentos y contador de pasos. El libro de Lumbre ya distingue esa reanudación de una carga de pesos. Mantén la misma distinción al usar otro framework.

Construir una comprobación útil

Exige que la pérdida final sea menor que una fracción pequeña de la inicial y que las predicciones se aproximen a los objetivos. Las dos condiciones detectan fallos distintos: un descenso pequeño no garantiza que hayas llegado a la solución, y una única predicción correcta no valida el conjunto.

CAPÍTULO 6

Una atención causal pequeña

6.1 Fijar las dimensiones

La práctica usa lote $B=1$, $H=2$ cabezas, $T=5$ posiciones y $D=4$ componentes por cabeza. Q , K y V tienen forma $[1,2,5,4]$. Multiplicamos Q por K transpuesta en sus dos últimos ejes y dividimos por la raíz de D , que aquí vale 2.

La matriz de puntuaciones tiene forma $[1,2,5,5]$. Cada fila corresponde a una consulta; sus columnas corresponden a posiciones de clave. La máscara causal prohíbe las columnas posteriores a la fila.

6.2 Enmascarar antes de softmax

Construimos una matriz booleana triangular superior, sin diagonal. En posiciones prohibidas colocamos menos infinito; conservamos las puntuaciones en las demás. Después aplicamos softmax sobre el último eje y multiplicamos por V .

```
scores = (tq @ tk.transpose(-1, -2)) / 2
mask = Tensor(np.triu(np.ones((5, 5), dtype=bool), 1),
              device=device)
weights = mask.where(float('-inf'), scores).softmax(axis=-1)
output = weights @ tv
```

Cada fila conserva al menos la diagonal. Una fila completamente enmascarada necesitaría una política explícita, porque restar el máximo de una fila de infinitos negativos puede producir una operación indeterminada.

6.3 Probar causalidad además de igualdad

La referencia NumPy resta el máximo por fila antes de exponenciar. Compara la salida completa con tinygrad. Luego aumenta en cien todos los componentes del último V y exige que las cuatro primeras posiciones no cambien. La última posición sí puede depender de ese valor.

Esta prueba ejerce la máscara de un forward con Q y K fijos. Un decoder completo también necesita verificar que sus representaciones no filtren información futura por otras rutas. No extrapoles una prueba local de atención a todas las capas de un modelo.

Cambiar la orientación de la máscara

Si enmascaras la parte inferior, la primera consulta podrá leer posiciones futuras. Describe qué comprobación de la práctica fallará. Después dibuja a mano la máscara de cinco por cinco y señala la diagonal que debemos conservar.

CAPÍTULO 7

Leer el compilador desde una expresión pequeña

7.1 Un experimento de inspección

Activa `DEBUG=2` al ejecutar el programa para observar información de ejecución de la revisión fijada. Con niveles mayores puedes obtener detalles de programas generados, pero la cantidad y el formato de salida dependen del código de esa revisión. Conserva los logs fuera del informe de tiempos.

La expresión `(x*2+1).relu()` es un buen inicio: tiene operaciones elemento a elemento y ninguna reducción. Identifica dónde se construyen sus UOps y qué programas aparecen al realizarla. No deduzcas el número de kernels contando operadores Python.

7.2 Un mapa de lectura del fuente

Abre primero `tinygrad/tensor.py` y sigue `realize`. Después estudia la planificación en `tinygrad/schedule/` y la ejecución en `tinygrad/engine/`. Los renderers se encuentran en `tinygrad/renderer/` y los backends en `tinygrad/runtime/`.

El commit usa UOps para representar diferentes niveles de trabajo. Por eso conviene anotar en cada lectura qué tipo de grafo estás viendo, qué invariantes exige su consumidor y qué transformación acaba de ejecutarse. La palabra «UOp» no basta para identificar una fase.

Para comparar con Lumbre, toma una sola operación y registra su forma, tipo, dispositivo, entradas y salida. Luego localiza el cálculo de índices del programa que la ejecuta. Un catálogo completo de clases antes de leer una operación suele ocultar esa relación.

7.3 Experimento de materialización

Compara una expresión encadenada con otra que realiza un intermedio explícito. Comprueba primero sus resultados y luego los programas emitidos. La materialización puede introducir almacenamiento y una frontera de ejecución. También puede ser necesaria si vas a reutilizar ese resultado.

Qué evidencia presentar

Entrega la expresión, el comando, el commit y el log. Señala qué observación apoya tu conclusión sobre fusión. Una captura de tiempo por sí sola no demuestra que dos operadores se hayan fusionado.

CAPÍTULO 8

TinyJit: repetir con entradas nuevas

8.1 El problema que resuelve la práctica

Una secuencia de operaciones puede repetir forma, tipos y estructura mientras cambian los datos. TinyJit permite capturar trabajo y volver a ejecutarlo. Usaremos una función que devuelve un Tensor realizado, sin extraer escalares dentro de la captura.

```
@TinyJit
def step(x):
    return (x*2+1).relu().realize()
```

En la implementación fijada, la primera llamada ejecuta normalmente, la segunda captura y las posteriores reutilizan la captura. Esa secuencia pertenece a este commit. El tutorial realiza varias llamadas antes de medir y no incluye captura en la mediana publicada. [4]

8.2 Evitar una prueba que solo repite el mismo resultado

Durante las cinco primeras llamadas usamos datos diferentes, almacenados en tensores nuevos de la misma forma y tipo. Cada salida se compara con su referencia correspondiente. Si el programa devolviese por accidente un resultado anterior, la comprobación lo detectaría.

Una captura puede reutilizar buffers de salida. Si necesitas conservar valores de varias llamadas, cópialos al almacenamiento apropiado antes de volver a ejecutar. En la prueba leemos y comprobamos cada salida antes de avanzar.

8.3 Lo que debe permanecer compatible

La revisión comprueba información de entradas al reutilizar la captura. Cambiar forma, tipo, dispositivo o estructura de argumentos requiere volver a estudiar su contrato. No supongas que un bucle o una condición Python basada en datos se volverá a evaluar dentro de un programa ya capturado.

Para una primera integración, fija las formas y pasa los datos variables como tensores. Prueba entradas distintas y conserva una función sin JIT como referencia. Después estudia las capacidades dinámicas de la revisión que utilices.

8.4 Medir una llamada caliente

La práctica prepara la entrada antes del reloj, sincroniza el dispositivo, llama a la función capturada y vuelve a sincronizar. Recoge treinta intervalos con `perf_counter`. La lectura de

salida para comparar ocurre después de la medida.

El intervalo incluye Python y sincronización del host. No equivale a un evento CUDA alrededor de un kernel ni al tiempo total de construir y copiar un batch. El tutorial CUDA utiliza otra frontera; mantén ambos informes separados.

Distintuir latencia y rendimiento

Repite el ensayo con 257 y con 65.537 elementos en dos capturas independientes. Explica por qué la relación de tiempos puede ser mucho menor que la relación de tamaños. El coste de llamada pesa más en el caso pequeño, pero la medición debe sostener esa interpretación.

CAPÍTULO 9

Diseñar tus siguientes experimentos

9.1 Un diagnóstico por capas

Si CPU pasa y CUDA falla, conserva exactamente las mismas entradas y separa forward, gradiente y actualización. Comprueba primero que se haya seleccionado CUDA y que el compilador del renderer exista. Un fallo al cargar una biblioteca no es una divergencia numérica.

Si el entrenamiento consume memoria creciente, revisa qué tensores guardas en listas y qué grafos siguen vivos. Si el tiempo medido parece casi cero, comprueba que estás esperando al trabajo y que no realizas repetidamente un tensor ya calculado.

9.2 Proyecto: clasificador de puntos

Genera puntos bidimensionales con una semilla fija y etiqueta por el signo de una combinación lineal que conozcas. Divide entrenamiento y evaluación antes de entrenar. Implementa una proyección de dos entradas a dos logits y una pérdida adecuada; comprueba un gradiente en un lote pequeño antes de ampliar el entrenamiento.

Conserva pesos, configuración y métricas por separado. Repite el forward en CPU y CUDA con los mismos pesos y datos. Empieza sin JIT; después captura el paso y exige que entradas nuevas produzcan los resultados esperados. No confundas una disminución de pérdida sobre entrenamiento con exactitud en el conjunto reservado.

9.3 Solución de organización

Separa generación de datos, modelo, paso de actualización y evaluación. Mantén los tensores entrenables en una lista explícita para el optimizador. La evaluación debe usar datos reservados y registrar su propia métrica. Guarda la semilla y el criterio de partición junto a la configuración.

Para un primer contraste del JIT, compara una actualización desde los mismos pesos, con el mismo batch y el mismo estado del optimizador. Una historia de cien pasos puede divergir por redondeo y dificultar la localización del primer error.

Entrega final

Entrega los resultados de las cinco prácticas, una derivación manual de gradiente, la comprobación de causalidad y una explicación de la frontera temporal del JIT. Añade un cambio propio con referencia. Debes poder identificar qué código es original del tutorial, qué comportamiento depende del commit de tinygrad y qué hardware has ejecutado.

CAPÍTULO 10

Resultados de esta edición

10.1 Diez grupos de prácticas superados

La campaña ejecutó cinco grupos en CPU y cinco en CUDA: arrays, gradientes, regresión, atención causal y TinyJit. Ambos dispositivos completaron todas las comprobaciones. El entorno utiliza Python 3.12.3, NumPy 2.3.5, Clang 18 y el commit de tinygrad indicado al comienzo. CUDA usa NVCC 12.9.86 y la RTX 5070 Ti Laptop bajo WSL2.

Dispositivo	Pérdida inicial	Pérdida tras 99 actualizaciones	Peso final	Bias final
CPU	3.4375	1.87062e-13	2.999999	-0.5
CUDA	3.4375	1.87062e-13	2.999999	-0.5

Se realizaron cien actualizaciones. La última pérdida de la historia se leyó antes de la actualización número cien; los pesos de la tabla son posteriores a ella. La recarga de pesos reprodujo exactamente las predicciones de cada backend dentro de su propia ejecución.

10.2 Comprobaciones numéricas y JIT

Dispositivo	Error del gradiente analítico	Error de atención frente a NumPy	Mediana JIT (us)
CPU	0	1.79e-07	134.792
CUDA	0	1.19e-07	121.169

El JIT se comprobó con cinco entradas diferentes antes de medir treinta llamadas calientes. Cada intervalo incluye llamada Python y sincronización; excluye transferencias de entrada y salida. La tabla acredita una medición local, no una comparación aislada de hardware. Los JSON conservan las muestras individuales.

Los informes de `tutorials/reports/tinygrad/CPU/` y `tutorials/reports/tinygrad/CUDA/` incluyen historias, errores y pesos. `tutorials/reports/campaign.json` registra el orden de ejecución y hashes del código. La atención comprueba un forward y causalidad respecto a V; no valida un decoder completo. La regresión comprueba aprendizaje de una función conocida, no calidad de un modelo de lenguaje.

CAPÍTULO A

Código completo de las prácticas

A.1 tutorials/tinygrad/labs.py

```
"""Original, deterministic lessons for tinygrad commit c3aec477b99d."""
import argparse
import json
from pathlib import Path
import time
import numpy as np
from tinygrad import Tensor, TinyJit, Device, Context, dtypes
from tinygrad.nn.optim import SGD

PIN = 'c3aec477b99d9bb87c54d91897cf60acd3f17441'

def check(actual, expected, atol=1e-5, rtol=1e-5):
    actual, expected = np.asarray(actual), np.asarray(expected)
    assert actual.shape == expected.shape
    assert np.isfinite(actual).all()
    np.testing.assert_allclose(actual, expected, atol=atol, rtol=rtol)
    return float(np.max(np.abs(actual-expected))) if actual.size else 0.0

def tensor(data, device):
    return Tensor(np.asarray(data, dtype=np.float32), device=device)

def arrays(device):
    a = np.arange(6, dtype=np.float32).reshape(2,3)
    x = tensor(a, device)
    bias = tensor([0.5, -1, 2], device)
    y = (x+bias).relu().sum(axis=1)
    error = check(y.numpy(), np.maximum(a+[0.5,-1,2],0).sum(axis=1))
    transposed = check(x.transpose().numpy(), a.T)
    for n in (1,127,128,129,257):
        v = np.arange(n,dtype=np.float32)/16
        check((tensor(v,device)*2-1).numpy(),v*2-1)
    return {'broadcast_error': error, 'transpose_error': transposed, 'device': str(x.
↪ device)}

def gradients(device):
    values = np.array([-2,-0.5,1,3],dtype=np.float32)
    x = tensor(values,device)
    loss = (x*x).mean()
    loss.backward()
    assert x.grad is not None
    analytical = 2*values/len(values)
    error = check(x.grad.numpy(),analytical)
    eps = 1e-3
    numerical=[]
    for i in range(len(values)):
```

```

        plus=values.astype(np.float64); minus=plus.copy()
        plus[i]+=eps; minus[i]-=eps
        numerical.append(((plus*plus).mean()-(minus*minus).mean())/(2*eps))
    fd_error = check(x.grad.numpy(),numerical,atol=1e-4)
    return {'loss': float(loss.item()), 'gradient': x.grad.numpy().tolist(),
            'analytical_error': error, 'finite_difference_error': fd_error}

def regression(device, output):
    inputs=np.linspace(-1,1,33, dtype=np.float32).reshape(-1,1)
    targets=3*inputs-0.5
    x,y=tensor(inputs,device),tensor(targets,device)
    w,b=tensor([[0]],device),tensor([0],device)
    optimizer=SGD([w,b],lr=0.2)
    history=[]
    with Context(TRAINING=1):
        for _ in range(100):
            optimizer.zero_grad()
            loss=((x@w+b-y)**2).mean()
            history.append(float(loss.item()))
            loss.backward()
            optimizer.step()
    prediction=(x@w+b).numpy()
    error=check(prediction,targets,atol=2e-4)
    output.mkdir(parents=True,exist_ok=True)
    np.savez(output/'regression_weights.npz',w=w.numpy(),b=b.numpy())
    with np.load(output/'regression_weights.npz') as saved:
        restored=(x@tensor(saved['w'],device)+tensor(saved['b'],device)).numpy()
    restore_error=check(restored,prediction,atol=0,rtol=0)
    assert history[-1]<history[0]*1e-5
    return {'steps':100, 'initial_loss':history[0], 'final_loss':history[-1],
            'w':w.numpy().tolist(), 'b':b.numpy().tolist(), 'max_error':error,
            'weights_reload_error':restore_error, 'history':history}

def attention(device):
    rng=np.random.default_rng(17)
    q,k,v=[rng.normal(size=(1,2,5,4)).astype(np.float32) for _ in range(3)]
    def forward(value):
        tq,tk,tv=[tensor(a,device) for a in (q,k,value)]
        scores=(tq@tk.transpose(-1,-2))/2
        mask=Tensor(np.triu(np.ones((5,5),dtype=bool),1),device=device)
        return mask.where(float('-inf'),scores).softmax(axis=-1)@tv
    scores=q@k.swapaxes(-1,-2)/2
    scores=np.where(np.triu(np.ones((5,5),dtype=bool),1),-np.inf,scores)
    p=np.exp(scores-scores.max(axis=-1,keepdims=True))
    p/=p.sum(axis=-1,keepdims=True)
    got=forward(v).numpy()
    error=check(got,p@v,atol=2e-5)
    changed=v.copy(); changed[:, :, 4, :]+=100
    future_error=check(forward(changed).numpy()[:, :, 4, :],got[:, :, 4, :],atol=2e-5)
    return {'shape':[1,2,5,4], 'numpy_error':error, 'future_value_error':future_error}

def jit(device):
    @TinyJit
    def step(x):
        return (x*2+1).relu().realize()
    # Each call gets fresh storage with the same shape and dtype.
    for i in range(5):

```

```

        data=np.arange(257,dtype=np.float32)/16+i
        check(step(tensor(data,device).realize()).numpy(),data*2+1)
x=tensor(np.arange(257,dtype=np.float32),device).realize()
samples=[]
for _ in range(30):
    Device[device].synchronize()
    start=time.perf_counter()
    result=step(x)
    Device[device].synchronize()
    samples.append(time.perf_counter()-start)
check(result.numpy(),np.arange(257,dtype=np.float32)*2+1)
return {'changed_inputs_checked':5,'samples_seconds':samples,
        'median_seconds':float(np.median(samples)),
        'scope':'warm TinyJit call plus host synchronization; excludes input/output
↳ transfers'}

if __name__=='__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('--device',choices=['CPU','CUDA'],required=True)
    parser.add_argument('--output',type=Path,required=True)
    args=parser.parse_args()
    results={'tinygrad_commit':PIN,'requested_device':args.device,'lessons':{}}
    for name,fn in [('arrays',arrays),('gradients',gradients),
                    ('regression',lambda d:regression(d,args.output)),
                    ('attention',attention),('jit',jit)]:
        results['lessons'][name]=fn(args.device)
    print(f'{args.device} {name}: PASS',flush=True)
    args.output.mkdir(parents=True,exist_ok=True)
    (args.output/'results.json').write_text(json.dumps(results,indent=2)+'\n',encoding='
↳ utf-8')

```

CAPÍTULO B

Fuentes y alcance

Las prácticas y explicaciones son originales. Los ejemplos se contrastan con el código de tinygrad fijado al commit `c3aec477b99d9bb87c54d91897cf60acd3f17441`. La documentación web puede cambiar; ante una diferencia de API, prevalece el fuente de esa revisión para reproducir esta edición.

Bibliografía

- [1] Tiny Corp y colaboradores. *tinygrad*, revisión del 30-09-2026. <https://github.com/tinygrad/tinygrad/tree/c3aec477b99d9bb87c54d91897cf60acd3f17441>.
- [2] tinygrad. *Tensor*: constructor, realización y backward. <https://github.com/tinygrad/tinygrad/blob/c3aec477b99d9bb87c54d91897cf60acd3f17441/tinygrad/tensor.py>.
- [3] tinygrad. Optimizadores y contexto de entrenamiento. <https://github.com/tinygrad/tinygrad/blob/c3aec477b99d9bb87c54d91897cf60acd3f17441/tinygrad/nn/optim.py>.
- [4] tinygrad. Captura y reutilización de TinyJit. <https://github.com/tinygrad/tinygrad/blob/c3aec477b99d9bb87c54d91897cf60acd3f17441/tinygrad/engine/jit.py>.
- [5] tinygrad. Selección de dispositivo, renderer e interfaz en helpers. <https://github.com/tinygrad/tinygrad/blob/c3aec477b99d9bb87c54d91897cf60acd3f17441/tinygrad/helpers.py>.
- [6] tinygrad. Documentación oficial y guía del runtime. <https://docs.tinygrad.org/>. <https://docs.tinygrad.org/developer/runtime/>.

Índice detallado

1.	De Lumbre a tinygrad	1
1.1.	Una revisión concreta	1
1.2.	La correspondencia que vamos a estudiar	1
2.	Instalar sin perder la versión	3
2.1.	Crear un entorno del tutorial	3
2.2.	CPU y CUDA son decisiones explícitas	3
2.3.	Leer la instalación antes de ejecutar	4
3.	Tensores: datos, forma y realización	5
3.1.	La primera fórmula	5
3.2.	Describir y ejecutar	5
3.3.	Transponer sin perder el contrato	5
4.	Gradientes que puedes comprobar a mano	7
4.1.	Una pérdida de cuatro variables	7
4.2.	Dos comprobaciones independientes	7
4.3.	Acumulación y grafos retenidos	7
5.	Entrenar una regresión y guardar sus pesos	9
5.1.	Un problema con respuesta conocida	9
5.2.	Seguir una actualización	9
5.3.	Derivar el primer paso	9
5.4.	Guardar, cargar y saber qué falta	10
6.	Una atención causal pequeña	11
6.1.	Fijar las dimensiones	11
6.2.	Enmascarar antes de softmax	11
6.3.	Probar causalidad además de igualdad	11
7.	Leer el compilador desde una expresión pequeña	13
7.1.	Un experimento de inspección	13
7.2.	Un mapa de lectura del fuente	13
7.3.	Experimento de materialización	13
8.	TinyJit: repetir con entradas nuevas	14
8.1.	El problema que resuelve la práctica	14
8.2.	Evitar una prueba que solo repite el mismo resultado	14
8.3.	Lo que debe permanecer compatible	14
8.4.	Medir una llamada caliente	14
9.	Diseñar tus siguientes experimentos	16
9.1.	Un diagnóstico por capas	16

9.2.	Proyecto: clasificador de puntos	16
9.3.	Solución de organización	16
10.	Resultados de esta edición	17
10.1.	Diez grupos de prácticas superados	17
10.2.	Comprobaciones numéricas y JIT	17
A.	Código completo de las prácticas	18
A.1.	tutorials/tinygrad/labs.py	18
B.	Fuentes y alcance	21